

ECE 251A: Digital Signal Processing I
Spring 2026
Prof. Piya Pal

Programming Assignment # 1
Due May 1, 11:59 pm

In this project, we will experimentally study the behavior of two ways to estimate power spectral density, one of which uses deep learning. You compare their performance and draw your own conclusions.

- 1. Periodogram Estimation.** Consider a Gaussian random process $y[n]$, which is generated as the output of a linear system with transfer function

$$H(z) = \frac{1 - 0.9z^{-1} + 0.81z^{-2}}{1 - 2.76z^{-1} + 3.809z^{-2} - 2.654z^{-3} + 0.924z^{-4}}$$

driven by zero mean, Gaussian white noise with variance 1.

- (a) What is the true autocorrelation function of the random process? Plot figures to visualize it.
- (b) Generate 1024 samples of this random process. Estimate the autocorrelation sequence using 64, 128, 256, 512, 1024 samples and compare with the true values. Comment on your results.
- (c) What is the true power spectrum of the random process? Show derivations. Plot figures to visualize it.
- (d) Use the previously generated 1024 samples, compute Periodogram estimates using 64, 128, 256, 512, 1024 samples and compare with the true spectrum.
- (e) Validate the statistical properties of the Periodogram estimate by generating P realizations of the random process and computing the sample mean and variance of the corresponding Periodogram estimate for the 5 data lengths mentioned above. You can experiment with P . Explain your observations.
- (f) Implement Welch periodogram estimate using a suitable window of your choice. Choose K , R , N_s appropriately for each of the 5 data lengths mentioned above. Validate its statistical properties and comment on your results.

2. Deep Learning Based Power Spectrum Estimation. Generate a white Gaussian (zero mean, unit variance) input sequence $x[n]$ with length $1024+P$ and reuse the generated 1024 samples in **Problem 1** as $y[n]$. Use the aforementioned data to learn a mapping:

$$y_{ML}[n] = f_{\theta}(x[n], x[n-1], \dots, x[n-P]) \approx y[n],$$

where $f_{\theta}(\cdot)$ is a neural network, whose weights are parameterized by θ . You will be training a neural network to “learn” the generative model of the WSS random process $y[n]$ and produce a random process $y_{ML}[n]$ that tries to approximate it.

Instructions:

- Please make sure you have installed Python (≥ 3.9), Numpy (≥ 1.22), Scipy (≥ 1.9) and Pytorch (≥ 2.0). For installing Numpy and Scipy, you can simply use “`pip install...`” (or “`conda install...`” if you are using Anaconda). For installing Pytorch, please follow the instructions <https://pytorch.org/get-started/locally/>.
- Save your data in the format of **x.npy**, **y.npy** (or **x.mat**, **y.mat**).
- You are given two modules in **Model_MLP.py** and **Model_LSTM.py** that implement a simple MLP and a simple LSTM respectively. The corresponding training codes are also given to you (i.e., **Train_MLP.py** and **Train_LSTM.py**). To train the models, choose your configurations: **P** (input dimension), **num-hidden-layers**, **hidden** (width of hidden layer), **epochs**, **lr** (learning rate), **seed**. Some examples of commands are shown below:

```
— python Train_MLP.py --data-dir . --P 16 --hidden 64 --num-hidden-layers 2
  --epochs 5000 --lr 1e-3 --batch-size 0 --seed 0

— python Train_LSTM.py --data-dir . --P 16 --num-layers 2 --hidden 32 --epochs
  5000 --lr 1e-3 --batch-size 0 --seed 0
```

- After training, you will find the model will be automatically saved as **Final_simpleMLP.pt** and **Final_simpleLSTM.pt**.

Tasks:

- (a) Use the prepared data to train your own MLP and LSTM models. Play with different training configurations and compare the convergence performance of MLP and LSTM.

- (b) Train the two models using three different values of P (note: for each P , you need to generate a new **x.npy** or **x.mat**). How does the choice of P affect performance?
- (c) Choose the most appropriate model configurations for MLP and LSTM and a value for P , and fix them. Re-generate $1024 + P$ new white Gaussian (zero mean, unit variance) samples $x[n]$ and save it as **x_new.npy** or **x_new.mat**. Input it to your pre-trained model (i.e., **Final_simpleMLP.pt** and **Final_simpleLSTM.pt**) to get $y_{ML}[n]$ by running **Infer_MLP.py** and **Infer_LSTM.py**. Repeat **Problem 1 (e)** using $y_{ML}[n]$. (Note: you can generate multiple realizations at once and save them to **x_new.npy** or **x_new.mat**.) Compare with what you observed in Problem 1 and discuss your results.
- (d) How would you estimate the impulse response using $x[n]$ and $y_{ML}[n]$? (Hint: sample cross-correlation.) Plot the estimated impulse response and compare it with the true impulse response as in **Problem 1 (a)**. Comment on your results.